



Mash-Up
iOS Team

2025.05.13

WWDC Study

고수가 되기 위해 땀땀 뽀개기

발표자

| 김나희 | 김남수 | 신상우 | 최혜린 | 임리나

Index

1 우리는 어떻게 했냐면요..~

2 나희 스터디 세션 발표

3 상우 스터디 세션 발표

4 남수 스터디 세션 발표

5 헤린 스터디 세션 발표

6 리나 스터디 세션 발표

우리는 어떻게 했냐면요 ..~

3/21 .. OT를 시작으로
각자 듣고 싶었던 WWDC 세션
을 정리하고 (격주로) 발표를 진
행 했어요 !

- (4월 2일부터) 매주 수요일 22시
◦ 2명 / 3명 격주로 발표 진행

4/2	남수,상우	
4/9	<u>나희,헤린</u>	리나: 환자
4/16	남수,상우,리나	나희: 지각
4/23	<u>나희,리나,헤린</u>	

이렇게 야무지게 ~
구글 폼으로 퀴즈도 만들어서 풀
었습니다?

Mergeable Libraries 퀴즈~

Link fast: Improve build and launch times 퀴즈

알차다 알차 ~

그렇게 저희는
총 10개의 세션을 스터디 했어요~

- Meet mergeable libraries
- Bring your app to Siri
- Demystify SwiftUI containers
- Discover Observation in SwiftUI
- Reduce networking delays for a more responsive app
- Beyond the basics of structured concurrency.
- Link fast: Improve build and launch times
- Run, Break, Inspect: Explore effective debugging in LLDB
- Catch up on accessibility in SwiftUI
- Get started with Dynamic Type

다들 세션 내용 궁금하시죠 ?

60초 요약 시작합니다.

세션 당 60초 요약이라..
인당 2분 정도만 쓸게요 ..?

나히

WWDC 2023: Meet mergeable libraries

Static Library

- 빌드 시 코드가 앱 바이너리에 복사됨
- 실행 시 외부 파일 필요 없음
- > 실행은 빠르지만 빌드 느림

Dynamic Library

- 바이너리는 앱과 별도로 존재
- 빌드 시 경로만 기록됨
- 실행 시 dyld가 디스크에서 찾아 로드
- > 빌드 빠르지만 실행은 느림

[그럼 Mergeable Library는?]

- 개발할 땐 동적처럼 나누고
- 빌드할 땐 정적처럼 병합
- 빌드/실행 모두 빠름

WWDC 2024: Bring your app to Siri

- Siri는 이제 Apple Intelligence라는 대규모 언어 모델(LLM) 기반으로 동작
- 기존엔 키워드 매칭 중심 → 이제는 자연어 해석 + 화면 문맥 이해까지 가능
- **App Intent Domain** = Apple이 정의한 기능별 도메인 그룹
 - iOS 18 기준, 12개 도메인 제공 (Mail, Photos 등 실사용 중)
 - Siri는 이 도메인을 기준으로 100개 이상의 액션을 지원
 - 앱이 제공하는 기능이 해당 도메인에 포함되면 자동으로 Siri가 연결 가능
- **Assistant Schema**
 - Swift 매크로를 이용하여 스키마 기반으로 Intent 정의
 - Siri가 앱을 제대로 이해할 수 있게 도와주면서, 개발자는 중복 없이 최소한의 코드로 Intent를 만들 수 있게 만듦
 - title/description 같은 반복적 메타데이터도 생략 가능 (스키마가 이미 알고 있음)
 - 개발자는 perform() 메서드 하나만 구현하면 됨!

WWDC 스터디

상우

WWDC 2024: Demystify SwiftUI containers

내용 1분 정리

WWDC23: Discover Observation in SwiftUI

내용 1분 정리

WWDC22: Reduce networking delays for a more responsive app

네트워크 대기 시간(latency)은 앱의 응답성에 가장 큰 영향을 미치는 요소입니다. 대역폭을 아무리 늘려도, 대기 시간이 길면 앱이 느려질 수밖에 없습니다. 실제로 대기 시간이 20ms씩 줄어들 때마다 페이지 로딩 속도가 선형적으로 빨라집니다.

이를 개선하기 위해서는 최신 프로토콜(HTTP/3, QUIC, IPv6, TLS 1.3) 적용이 중요합니다. Swift에서는 URLSession이나 Network.framework를 사용하면 자동으로 최신 프로토콜을 활용할 수 있습니다. 특히 HTTP/3는 zero RTT, 빠른 패킷 손실 복구, 네트워크 변경 시 연결 유지 등으로 응답성을 크게 높입니다.

또한, 서버의 버퍼 크기를 줄이면 오래된 데이터 뒤에 새로운 패킷이 대기하는 현상(버퍼블로트)을 줄여, 실시간 스트리밍 등에서 즉각적인 반응을 얻을 수 있습니다. Apple의 networkQuality 도구로 서버와 네트워크의 응답성을 측정하고, 필요시 서버 설정을 조정해야 합니다.

마지막으로, L4S(저지연, 저손실, 확장성 높은 네트워크) 기술이 도입되면서, 인프라가 지원된다면 별도의 코드 변경 없이도 대기 시간을 획기적으로 줄일 수 있습니다.

WWDC23: Beyond the basics of structured concurrency

구조화된 동시성(Structured Concurrency)은 Swift에서 동시성 코드를 안전하고 예측 가능하게 관리하는 핵심 패턴입니다.

async let, taskGroup을 사용하면 작업의 생성과 종료가 명확하게 스코프에 묶여, 작업의 수명과 취소 시점을 쉽게 추론할 수 있습니다.

비구조화된 Task, Task.detached는 명시적으로 관리해야 하므로, Swift에서는 구조화된 작업 사용을 권장합니다.

작업 취소(Cancellation)는 부모 작업이 취소되면 자식 작업도 함께 취소 신호를 받지만, 실제 취소는 코드에서 isCancelled, checkCancellation 등으로 직접 확인해야 합니다.

AsyncSequence 등에서는 withTaskCancellationHandler로 취소 이벤트를 감지할 수 있습니다.

작업 우선순위(Task Priority)는 시스템이 더 중요한 작업을 먼저 실행하도록 돕습니다.

자식 작업은 부모의 우선순위를 상속받고, 우선순위 역전 현상도 자동으로 관리됩니다.

작업이 끝날 때마다 리소스를 즉시 해제하는 discardTaskGroup을 사용할 수 있습니다.

discardTaskGroup은 결과를 모으지 않아 메모리 효율이 뛰어나고, 오류 발생 시 형제 작업도 자동 취소됩니다.

TaskLocal 값은 작업 계층 구조에만 영향을 주는 로컬 데이터로, 로깅 등에서 작업별 컨텍스트 정보를 안전하게 전파할 수 있습니다.

헤린

WWDC22: Link fast: Improve build and launch times

[앱의 빌드 및 런타임 링킹 성능을 개선하는 방법]
(링킹의 내부 동작 & 이를 최적화하는 방법)

- 링킹 프로세스의 기본 개념과 이것이 앱 빌드 파이프라인에서 어떤 역할을 하는지(링커가 어떻게 여러 객체 파일과 라이브러리를 하나의 실행 가능한 바이너리로 통합하는지)
- 다양한 링킹 옵션들에 대해 알아보기. 정적 링킹과 동적 링킹의 차이점, 그리고 각각이 앱 성능에 어떤 영향을 미치는지
- 최신 업데이트와 도구들이 어떻게 링킹 성능을 향상시키는지

링커의 고수가 되고 싶다? 저희 레포로 놀러오세요...🧙‍♂️

WWDC24: Run, Break, Inspect: Explore effective debugging in LLDB

[기존의 단순한 Logging과 print debugging을 넘어서 LLDB를 적극 활용한 디버깅 방식에 대한 섹션]
(LLDB: Xcode와 함께 제공되는 디버깅 툴)

- Crashlog를 활용해서 크래시 났을 때 상태로 프로젝트 맥락 복구 시키기
- 백트레이스를 통해 크래시가 발생한 맥락 파악하기
- breakpoint 활용하기(브레이크포인트 호출 경로 필터링하기, command line으로 break point 생성 및 사용하기, 특정 조건에서만 브레이크 포인트 활성화시키기 등...)
- Swift 6의 새로운 "p" 명령어 활용해서 프로그램 상태 알아보기

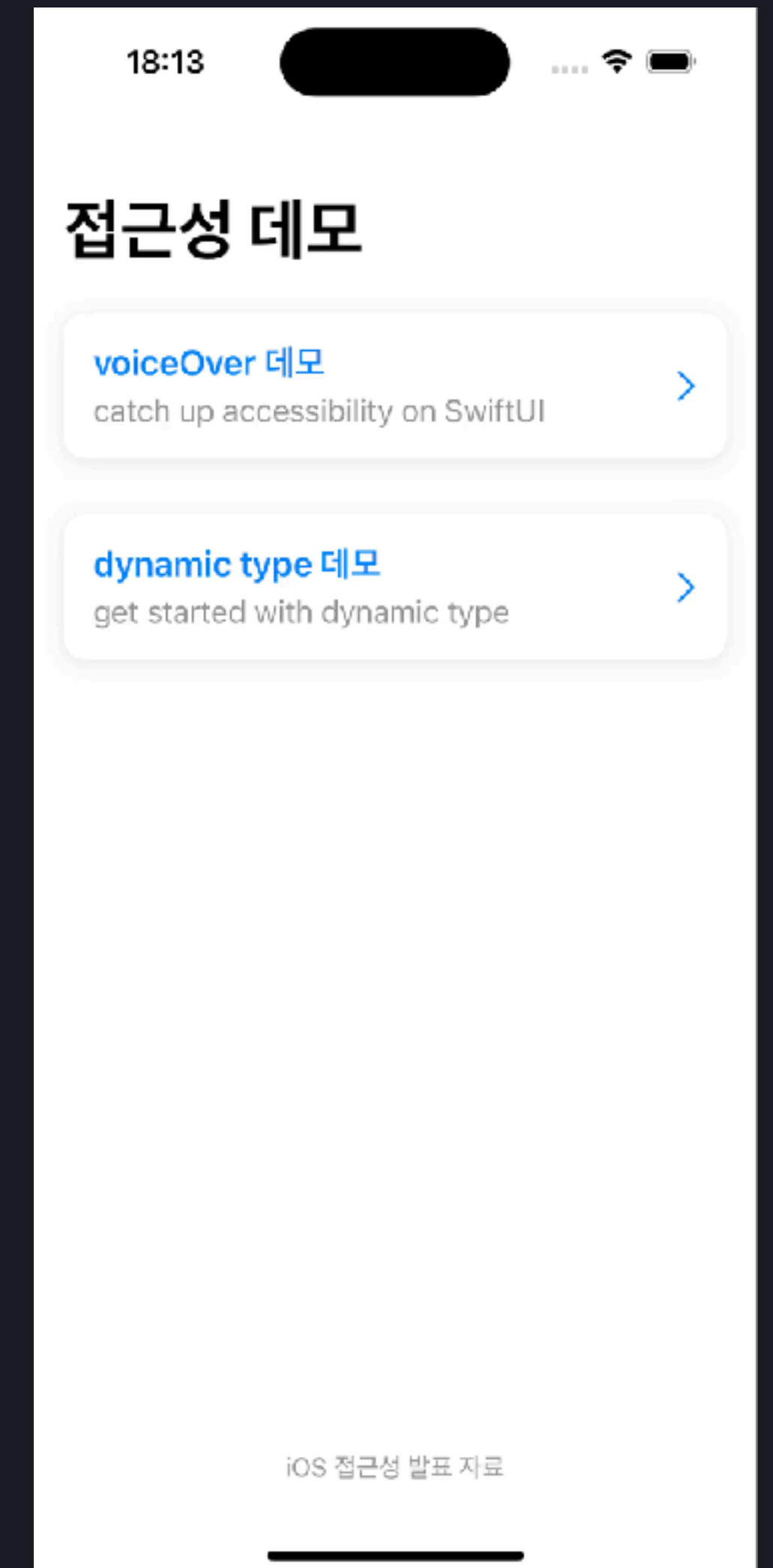
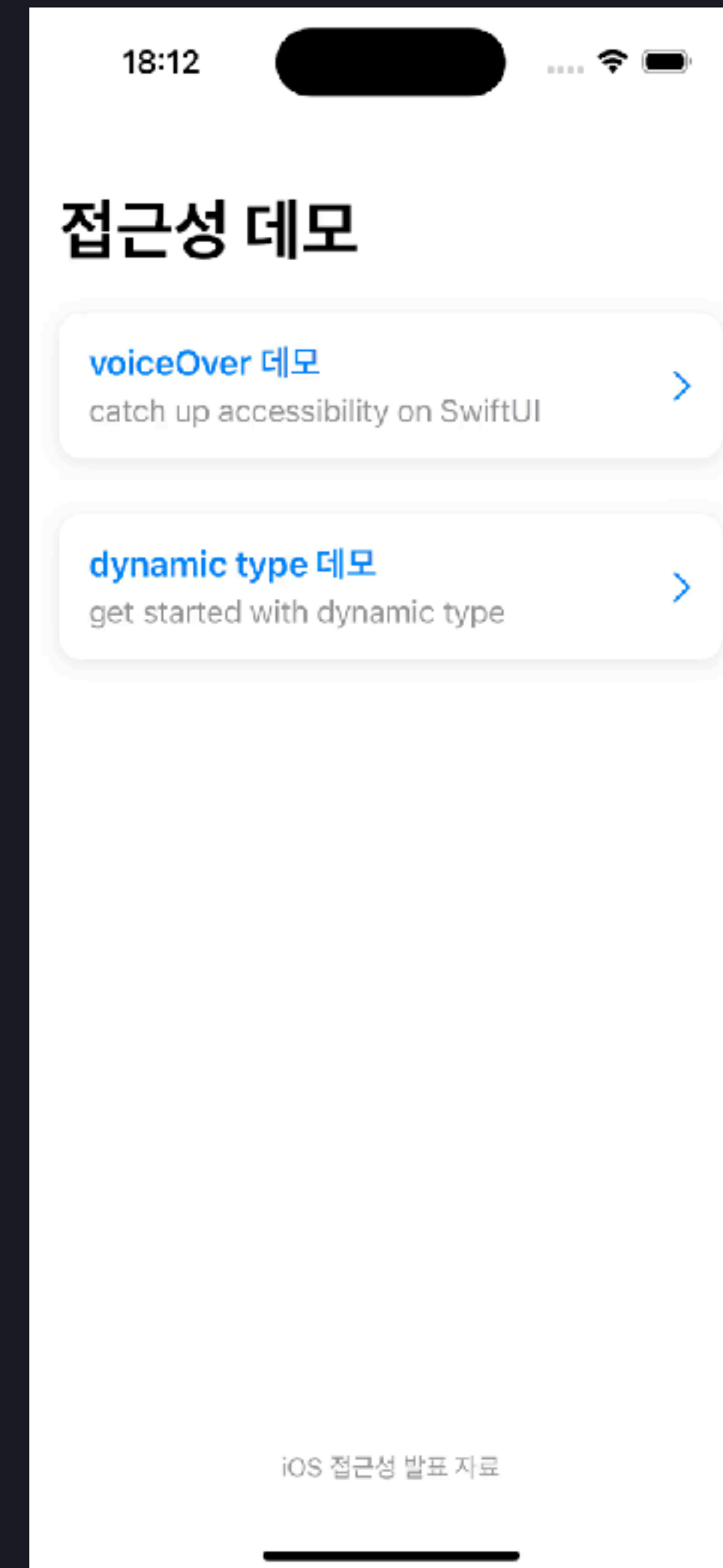
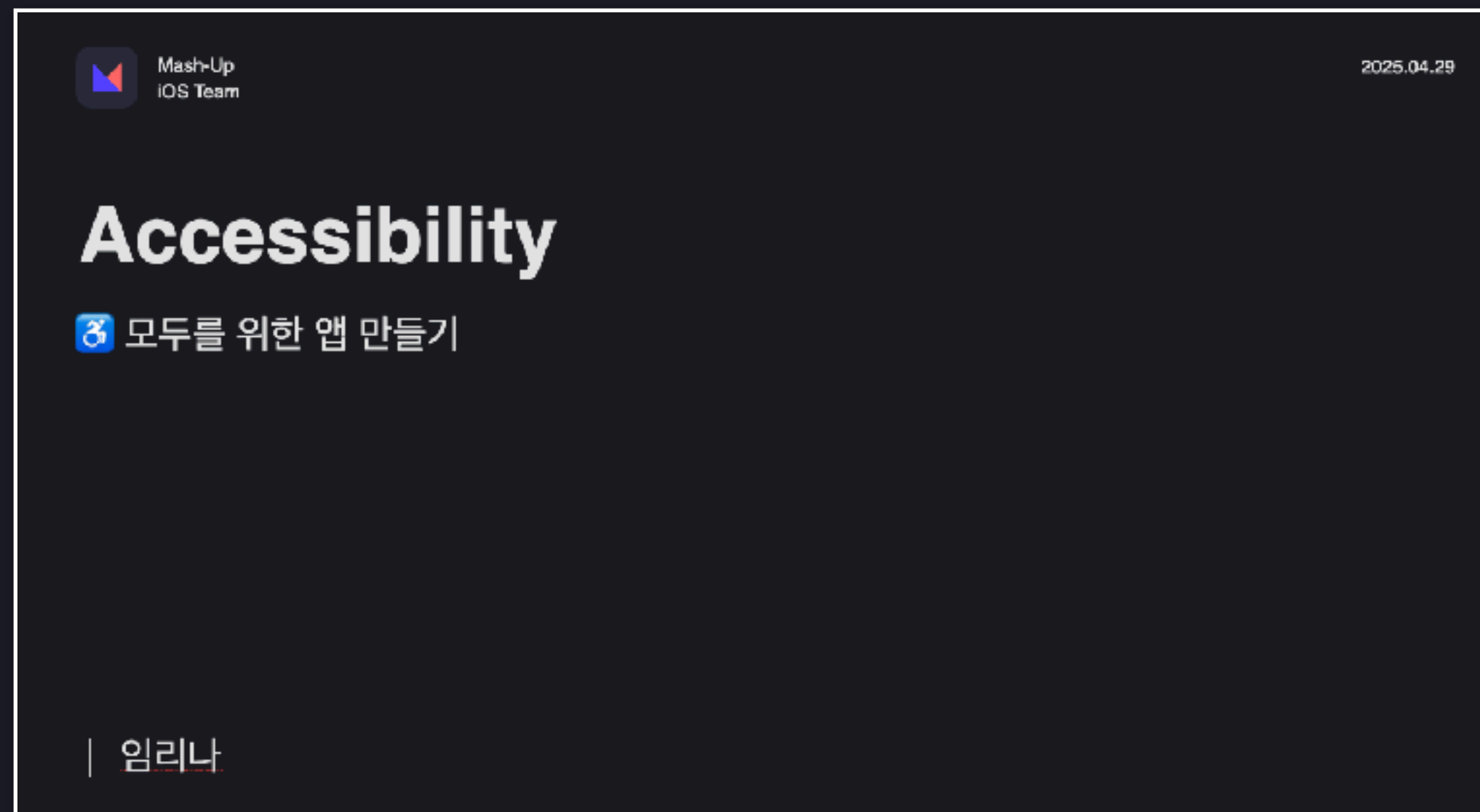
디버깅의 고수가 되고 싶다? 저희 레포로 놀러오세요...🧙‍♂️

WWDC 스터디

리나

WWDC24: Catch up on accessibility in SwiftUI

WWDC24: Get started with Dynamic Type

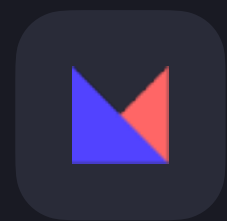


각 세션에 대해 더 자세히 알고 싶다면

<https://github.com/mash-up-kr/WWDC-Study>

해당 레포의 발표 자료와

WWDC 세션 영상 참고 바랍니다 ^_^



Mash-Up
iOS Team

2025.05.13

Thank you.

발표자

| 김나희 | 김남수 | 신상우 | 최혜린 | 임리나